
AN0005- EC NB-IoT LwM2M 应用笔记

EigenCOMM Wireless Microcontroller

文档说明

本文旨在帮助用户快速通过 AT 命令，使用 LwM2M 协议，将自己的设备与 LwM2M server 连接。

目录

1. 引言	4
1.1 文档目的	4
1.2 LwM2M 协议简介	4
1.3 内容简介	5
2. LwM2M Server	5
2.1 非加密客户端	5
2.2 加密客户端	5
3. 应用	7
3.1 设备对象选择	7
3.2 设备上线	7
3.3 指令操作流程	9
3.3.1 读写执行指令	9
3.3.2 Observe 与 Notify 指令	10
3.4 LwM2M Commands	11
3.4.1 AT+LWM2MCREATE	11
3.4.2 AT+LWM2MDELETE	12
3.4.3 AT+LWM2MADDOBJ	13
3.4.4 AT+LWM2MDELOBJ	13
3.4.5 +LWM2MREAD	14
3.4.6 +LWM2MWRITE	14
3.4.7 +LWM2MEXECUTE	15
3.4.8 +LWM2MOBSERVE	16
3.4.9 AT+LWM2MREADCONF	16
3.4.10 AT+LWM2MWRITECONF	17
3.4.11 AT+LWM2MEXECUTECONF	18
3.4.12 AT+LWM2MNOTIFY	19
3.4.13 AT+LWM2MUPDATE	20
3.4.14 <err>说明	20
4. 附录	21
4.1 上电检查流程	21
4.2 模组侧设备登陆流程	21
4.3 读写执行流程	21
4.4 客户端侧注销流程	21
5. 版本	22
6. 关于我们	错误!未定义书签。

EiGENCOMM CONFIDENTIAL

1. 引言

1.1 文档目的

EC NB-IoT LwM2M 应用笔记描述了如何使用 LwM2M AT 指令，实现客户的设备作为 LwM2M 客户端，登陆公开的 LwM2M 服务器，并从服务器网页上实现对设备的监控。

1.2 LwM2M 协议简介

LwM2M 是一套适用于物联网的协议。全称是 Lightweight Machine-To-Machine，由 OMA(Open Mobile Alliance)提出并定义。协议的主要实体包括 LwM2M Server，LwM2M Client 和 LwM2M Bootstrap Server。LwM2M Server 作为服务器，部署在服务提供商处，LwM2M Client 作为客户端，部署在各个 LwM2M 设备上，此外根据需要还可以部署 LwM2M Bootstrap Server，对客户端进行引导。LwM2M 的基本架构如下图所示

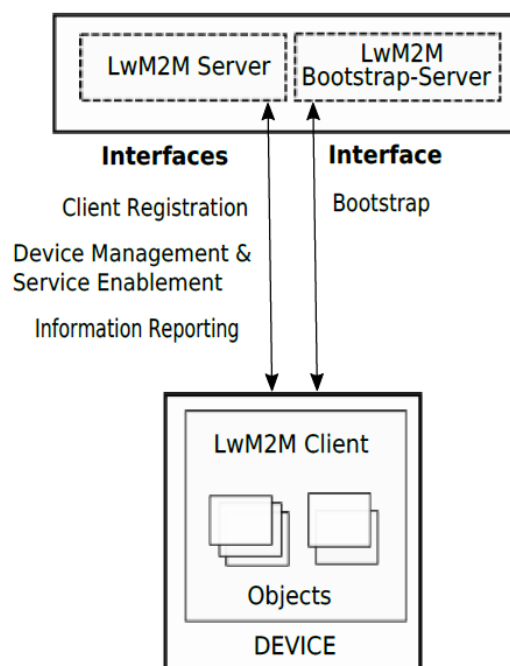


图 1：LwM2M 协议框架

在这些实体之间，协议定义了 4 个接口，来实现各自的功能。

1、引导接口（Bootstrap）

LwM2M 引导服务器向 LwM2M 客户端提供注册到 LwM2M 服务器的必要信息。

2、客户端注册接口（Client Registration）

使 LwM2M 客户端与 LwM2M 服务器互联，并告知 LwM2M 客户端的相关信息。

3、设备管理与服务实现接口（Device Management and Service Enablement）

LwM2M 服务器向 LwM2M 客户端发送指令，LwM2M 客户端对指令做出回应并将回应消息发送给 LwM2M 服务器

4、信息上报接口（Information Reporting）

该接口允许 LwM2M 服务器向 LwM2M 客户端订阅资源信息，LwM2M 客户端接收订阅后按照约定的模式向服务端报告自己的资源变化情况。

LwM2M 协议基于 REST 架构。协议的消息传递通过 CoAP 协议达成，消息的加密由 DTLS 负责。此外，LwM2M 定义了以资源（Resource）为基本单位的模型，每个资源表示 LwM2M 客户端中的一项可用信息。资源

都存在于对象实例中（Object Instance），即对象（Object）的实例化。协议允许使用者根据实际需要自定义对象来描述自己的产品。更详细的 LwM2M 协议描述请访问 <https://www.omaspecworks.org>。

1.3 内容简介

本文介绍了 eclipse 提供的 LwM2M 服务器的使用方法。该服务器主要供 LwM2M 的开发者测试所用，只是提供了遵循标准 LwM2M 协议的一些基本功能。客户产品最终商用时，需要开发符合自己需求的 LwM2M 服务器，或者使用其他商用 LwM2M 服务器。

本文还介绍了 LwM2M 客户端的注册流程，各种指令的操作流程，相关 AT 指令的参数的详细解释。最后提供了一点低功耗的使用建议。

2. LwM2M Server

常用的 LwM2M 服务器地址为 `coap://leshan.eclipseprojects.io`。它是 Eclipse Leshan 提供的一个公开的 LwM2M 服务器，提供加密(端口号 5684)和非加密（端口号 5683）两种客户端接入方式。可供用户测试自己 LwM2M 客户端。此外，还提供了一个 https 服务器：<https://leshan.eclipseprojects.io>，用户可以在网页上对自己的 LwM2M 客户端进行读写等操作。

由于此公开服务器时常关闭或是无法连接。本文另外提供了一个 LwM2M 服务器用于测试：`coap://Leshan.eigencomm.com`。同样它也提供一个 http 服务器：`http://leshan.eigencomm.com`。本文提供的例子均连接此服务器。

2.1 非加密客户端

如果是非加密客户端，服务器上不需要配置，直接注册客户端即可。

2.2 加密客户端

如果是客户端想通过 DTLS 方式连接服务器，需要点击网页上的 SECURITY 页面，为自己的客户端添加安全配置，如下图所示。

注：本文对加密客户端未有充分测试，故列举的例子都为非加密客户端

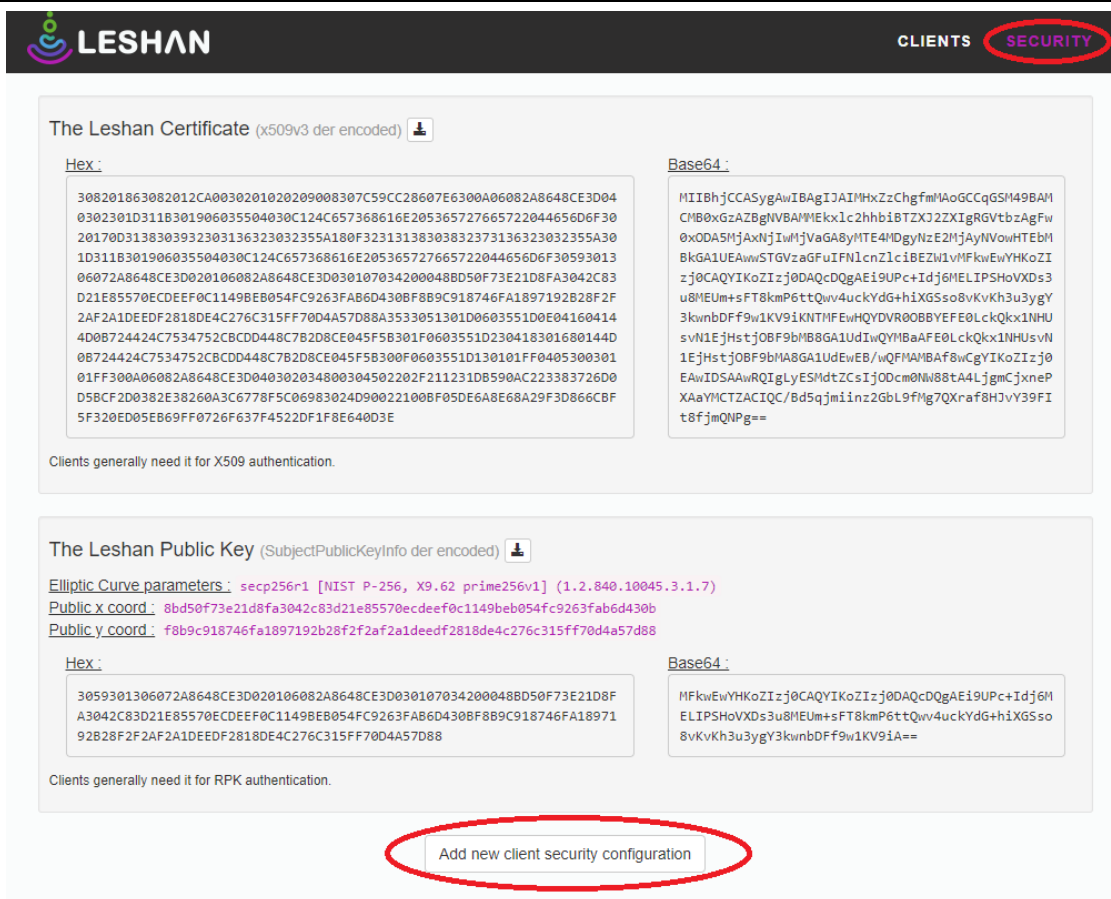


图 2：添加客户端安全配置

例如客户端要采用预共享密钥的加密方式，点击上图中的 add new client security configuration 添加。Identity 可以设为自己产品的名称。Key 可以设为设备的 IMEI 号。

New security configuration

Client endpoint	ec-client-116
Security mode	Pre-Shared Key
Identity	eigencomm
Key	323031393031

Hexadecimal format

Close Create

图 3：新的安全配置内容

添加完成后，可在列表中看到：

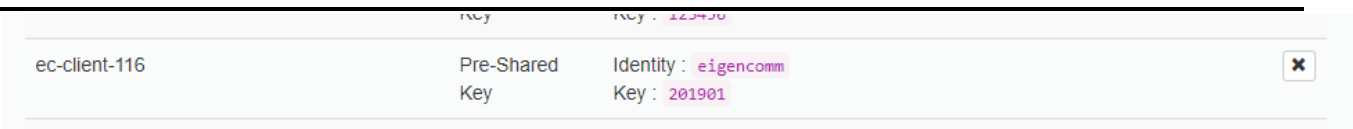


图 4：安全配置

3. 应用

3.1 设备对象选择

目前标准 object 定义主要由 IPSO 联盟和 OMA 定义，参见 www.omaspecworks.org。如果使用非标准的自定义的 object id，初次进入本文使用的 LwM2M 服务器的网页时只能显示 object 的 id 和实例，对整个实例进行读取后，才能显示 object 的 resource id，然后才能对 resource id 进行操作。想要自定义新的 object，可访问 <http://www.openmobilealliance.org/wp/OMNA/LwM2M/LwM2MRegistry.html>

3.2 设备上线

本文中的设备为非加密设备。这里通过 AT 命令的方式，操作设备上线。

设备上线的步骤如下：

- 1、模组上电后，从串口输入 ATQ0。确保主动上报消息能从串口输出。
- 2、创建 LwM2M 客户端实例，向 LwM2M 服务器注册。AT+LWM2MCREATE="Leshan.eigencomm.com",5683,56830,"ec-lient-128",60。

成功后串口会输出+LWM2MCREATE:0

OK

失败返回+LWM2M ERROR:XXX。详细参考 AT Manual.docx。

该条指令执行成功后，网页的 clients 页面能够看到设备：

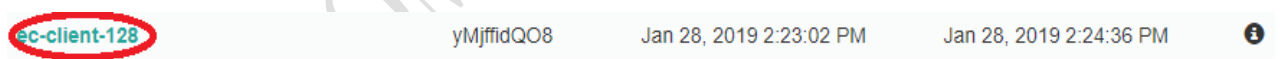


图 5：设备上线

3、添加对象实例 object id:3303，

在添加对象实例前。设备有默认的两个 object，即 id 为 1 的 LwM2M Server 和 id 为 3 的 Device。它们是 OMA 定义的标准属性。当前属性值都为默认值，如下图所示，如果客户希望使用这两个标准 object，需要提特殊需求。

LwM2M Server /1

Create New Instance

Attribute	Path	Observe	Read	Write	Delete	Value
Instance 0	/1/0	Observe ▶	Read	Write	Delete	
Short Server ID	/1/0/0	Observe ▶	Read	Write	Delete	123
Lifetime	/1/0/1	Observe ▶	Read	Write	Delete	60
Default Minimum Period	/1/0/2	Observe ▶	Read	Write	Delete	0
Default Maximum Period	/1/0/3	Observe ▶	Read	Write	Delete	0
Disable	/1/0/4	Observe ▶	Read	Write	Delete	0
Disable Timeout	/1/0/5	Observe ▶	Read	Write	Delete	false
Notification Storing When Disabled or Offline	/1/0/6	Observe ▶	Read	Write	Delete	U
Binding	/1/0/7	Observe ▶	Read	Write	Delete	
Registration Update Trigger	/1/0/8	Observe ▶	Read	Write	Delete	

Device /3

Create New Instance

Attribute	Path	Observe	Read	Write	Delete	Value
Instance 0	/3/0	Observe ▶	Read	Write	Delete	
Manufacturer	/3/0/0	Observe ▶	Read	Write	Delete	Open Mobile Alliance
Model Number	/3/0/1	Observe ▶	Read	Write	Delete	Lightweight M2M Client
Serial Number	/3/0/2	Observe ▶	Read	Write	Delete	345000123
Firmware Version	/3/0/3	Observe ▶	Read	Write	Delete	1.0
Reboot	/3/0/4	Observe ▶	Read	Write	Delete	
Factory Reset	/3/0/5	Observe ▶	Read	Write	Delete	
Available Power Sources	/3/0/6	Observe ▶	Read	Write	Delete	0=1, 1=5
Power Source Voltage	/3/0/7	Observe ▶	Read	Write	Delete	0=3800, 1=5000
Power Source Current	/3/0/8	Observe ▶	Read	Write	Delete	0=125, 1=900
Battery Level	/3/0/9	Observe ▶	Read	Write	Delete	100
Memory Free	/3/0/10	Observe ▶	Read	Write	Delete	15
Error Code	/3/0/11	Observe ▶	Read	Write	Delete	0=0
Reset Error Code	/3/0/12	Observe ▶	Read	Write	Delete	
Current Time	/3/0/13	Observe ▶	Read	Write	Delete	2013-05-02T18:47:33+08:00
UTC Offset	/3/0/14	Observe ▶	Read	Write	Delete	+01:00
Timezone	/3/0/15	Observe ▶	Read	Write	Delete	Europe/Berlin
Supported Binding and Modes	/3/0/16	Observe ▶	Read	Write	Delete	U
Device Type	/3/0/17	Observe ▶	Read	Write	Delete	
Hardware Version	/3/0/18	Observe ▶	Read	Write	Delete	
Software Version	/3/0/19	Observe ▶	Read	Write	Delete	
Battery Status	/3/0/20	Observe ▶	Read	Write	Delete	
Memory Total	/3/0/21	Observe ▶	Read	Write	Delete	
ExtDevInfo	/3/0/22	Observe ▶	Read	Write	Delete	

图 6：默认 object

添加设备 object，以温度传感器 3303 为例，设备有 5601，5602，5700，5701 四个属性
AT+LWM2MADDOBJ=0,3303,0,4,"5601;5602;5700;5701"

Temperature /3303

Create New Instance

Attribute	Path	Observe	Read	Write	Delete
Instance 0	/3303/0	Observe ▶	Read	Write	Delete
Min Measured Value	/3303/0/5601	Observe ▶	Read	Write	Delete
Max Measured Value	/3303/0/5602	Observe ▶	Read	Write	Delete
Min Range Value	/3303/0/5603	Observe ▶	Read	Write	Delete
Max Range Value	/3303/0/5604	Observe ▶	Read	Write	Delete
Reset Min and Max Measured Values	/3303/0/5605	Observe ▶	Read	Write	Delete
Sensor Value	/3303/0/5700	Observe ▶	Read	Write	Delete
Sensor Units	/3303/0/5701	Observe ▶	Read	Write	Delete

图 7：添加的 object

刷新页面后，可看到添加的 object。虽然只在客户端添加了 4 个属性值，但服务器仍然会显示 OMA 定义的这个 object 的所有属性值。

3.3 指令操作流程

3.3.1 读写执行指令

从服务器上点击读取属性 5601。服务器向设备发送 read 指令。模组会从串口显示主动上报的 AT 指令：
+LWM2MREAD:0,3303,0,5601。目前 leshan 服务器的指令有 5 秒的超时，在 5 秒之内没有收到回复即提示超时。设备应发送读指令的响应，例：AT+LWM2MREADCONF=0,3303,0,5601,3,4,"3.14"。（AT 的详细说明参见“AT Manual.docx”或 3.4 节）。

服务器收到响应后会在页面上显示出来。



图 7：读取资源

写指令和执行指令和读取指令类似，命令流程详见下图

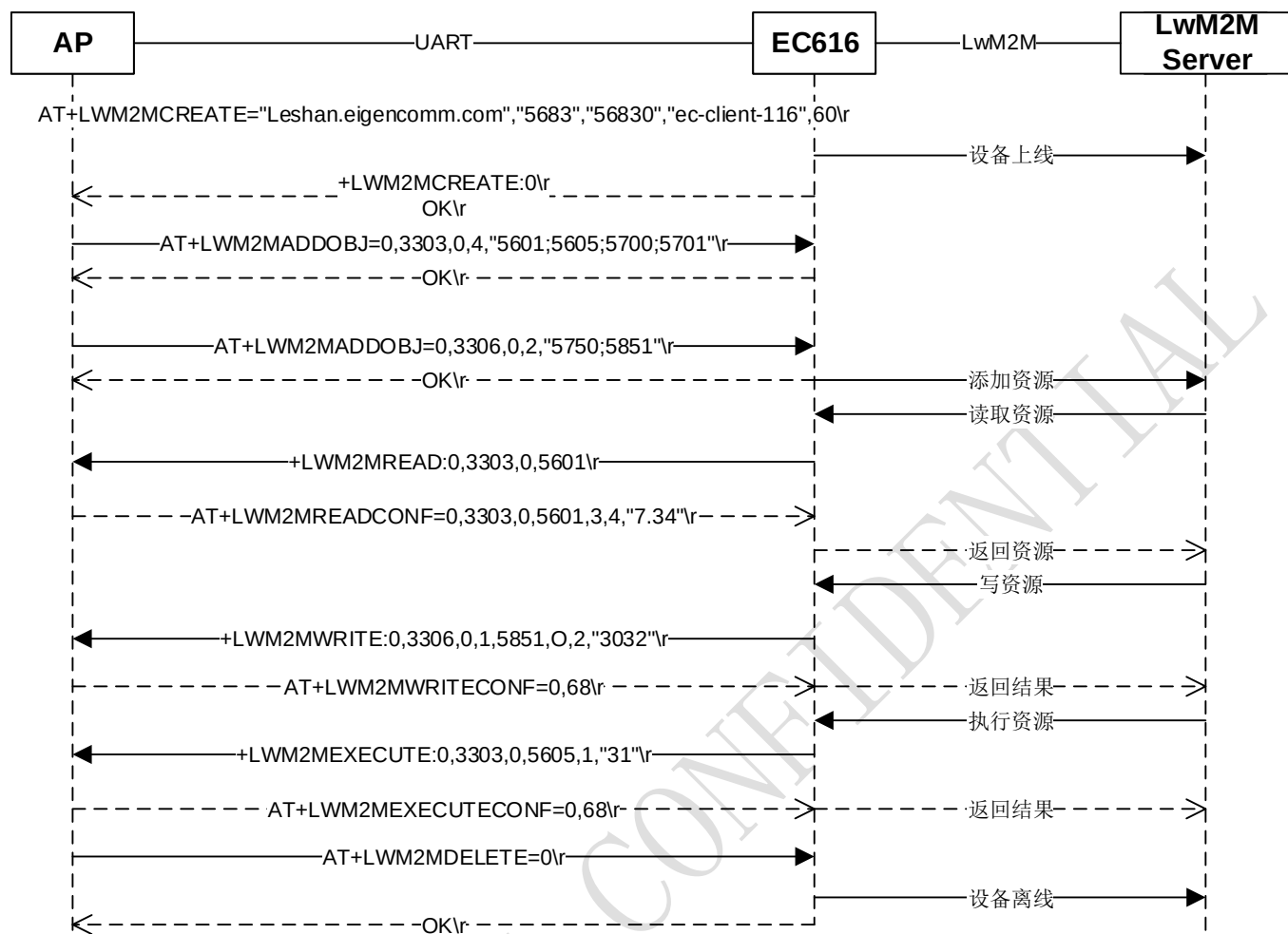


图 8: AT 流程

3.3.2 Observe 与 Notify 指令

在网页上点击 **observe** 指令，如下图所示，表示服务器想要观察 **object:5601**。在 **lwm2m** 协议的设计里，可以对被观察对象的资源设置各种阈值，当资源的阈值达到时，客户端即将该对象的数值上报。如果点击 **observe**，客户端会不断读取资源值判断资源是否达到阈值。目前 **leshan** 的这个 **https** 服务器没有给资源设置阈值的功能，而且由于本文采取的是 **AT** 的方式与客户 **MPU** 交互，考虑到低功耗的需求，频繁读取客户 **MPU** 会增加功耗。所以本文采取用户 **MPU** 发 **notify** 指令来通知服务器，**notify** 指令有两种使用方式，一种是当某资源值有变化，发送 **AT+LWM2MNOTIFY=<clientId>,<objectId>,<instanceId>,<resourceId>** 给客户端，然后客户端发出读资源的 **AT** 指令，客户 **MPU** 响应指令后，客户端再将资源值发给服务器。这种方式多一次 **at** 交互。另一种方式是，用户 **MPU** 直接在 **notify** 指令携带已变化的资源值 **AT+LWM2MNOTIFY=<clientId>,<objectId>,<instanceId>,<resourceId>,<valuetype>,<valuelen>,<value>**。客户端直接将资源值发给服务器。



图 9: observe 资源

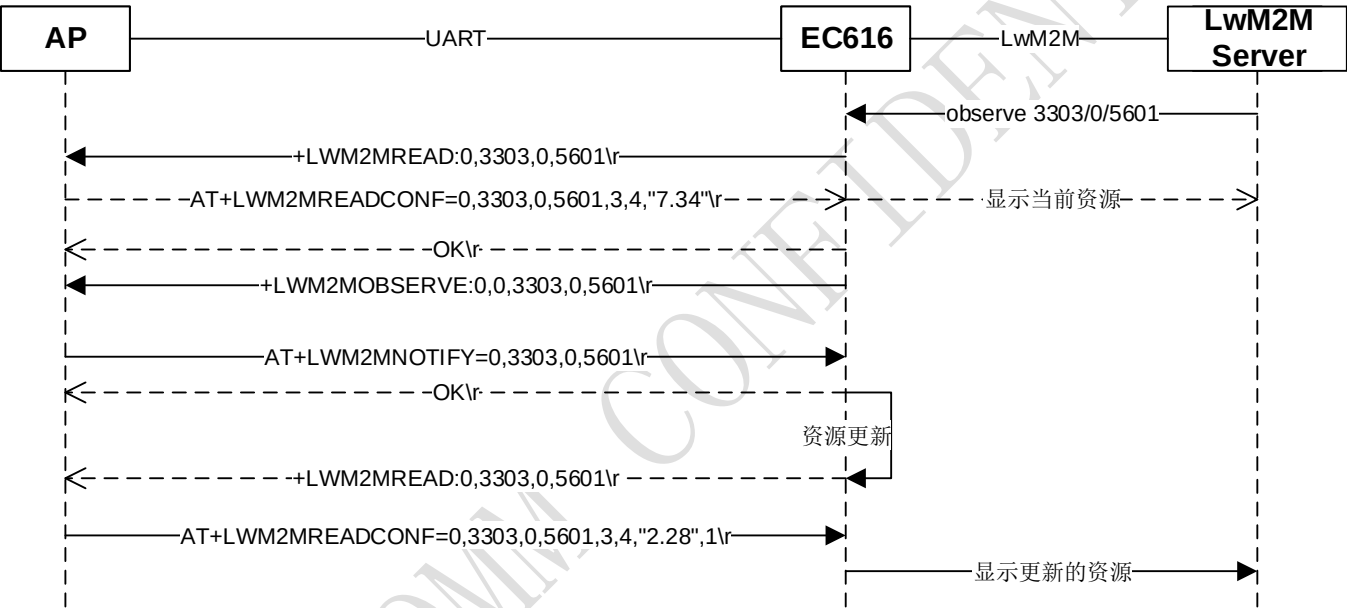


图 10: observe 和 notify（方式一）的 AT 流程

3.4 LwM2M Commands

3.4.1 AT+LWM2MCREATE

该指令创建一个 LwM2M 客户端的实例。指定 LwM2M 服务端地址、端口、客户端名称、保活时间，如果使用 PSK 加密方式，该指令还要指定 psk identity 和 psk key。

设置指令 AT+LWM2MCREATE=<server>,<port>,<local port>,<endpoint>,<life time>[,<psk id>,<psk>]	回复: +LWM2MCREATE: <clientId> OK 如果发生错误, 回复: + LWM2M ERROR: <err>
测试指令	回复:

AT+LWM2MCREATE=?	+LWM2MCREATE: "<server>",(range of supported< port >),(range of supported< local port >),"<endpoint>",(range of supported< lifetime >),"<psk_id>","<psk>" OK
------------------	--

参数

<clientId>	整型 LwM2M客户端序号
<server>	字符型 LwM2M服务器的域名或IP地址
<port>	整型 LwM2M服务器的端口
<local port>	整型 LwM2M客户端的本地端口
<endpoint>	字符型 LwM2M客户端的名称
<life time>	整型 LwM2M客户端的保活时间
<psk id>	字符型 LwM2M客户端的psk标识
<psk>	字符型 LwM2M客户端的psk密钥

3.4.2 AT+LWM2MDELETE

该指令销毁一个 LwM2M 客户端实例。

设置指令 AT+LWM2MDELETE=<clientId>	回复: OK 如果发生错误, 回复: + LWM2M ERROR: <err>
测试指令 AT+LWM2MDELETE=?	回复: +LWM2MDELETE: (list of supported<clientId>) OK

参数

<clientId>	整型 +LWM2MCREATE指令返回的LwM2M客户端序号
------------	-----------------------------------

3.4.3 AT+LWM2MADDOBJ

此命令将LwM2M对象添加到指定的LwM2M客户端实例，它指定对象id，实例id和资源id。

设置指令 AT+LWM2MADDOBJ=<clientId>,<objectId>,<instanceId>,<resourceCount>,<resourceIds>	回复: OK 如果发生错误, 回复: +LWM2M ERROR: <err>
测试指令 AT+LWM2MADDOBJ=?	回复: +LWM2MADDOBJ: (list of supported<clientId>),(range of supported<objectId>),(range of supported<instanceId>),(range of supported<resourceCount>),"<resourceIds>" OK

参数

<clientId>	整型 +LWM2MCREATE指令返回的LwM2M客户端序号
<objectId>	整型 对象id
<instanceId>	整型 实例id
<resourceCount>	整型 资源数目
<resourceIds>	字符型 用分号隔开的资源id

例子:

AT+LWM2MADDOBJ=0,3306,111,3,"5750;5850;5851"

3.4.4 AT+LWM2MDELOBJ

该指令删除 LwM2M 客户端的特定对象

设置指令 AT+LWM2MDELOBJ=<clientId>,<objectId>	回复: OK 如果发生错误, 回复: +LWM2M ERROR: <err>
测试指令 AT+LWM2MDELOBJ=?	回复: +LWM2MDELOBJ: (list of supported<clientId>),(range of supported<objectId>)

OK

参数

<clientId>	整型 +LWM2MCREATE指令返回的LwM2M客户端序号
<objectId>	整型 对象id

3.4.5 +LWM2MREAD

表示LwM2M客户端收到的读请求的主动上报消息。

+LWM2MREAD: <clientId>,<objectId>,<instanceId>,<resId>

参数

<clientId>	整型 +LWM2MCREATE指令返回的LwM2M客户端序号
<objectId>	整型 想要读取的对象id
<instanceId>	整型 想要读取的实例id
<resId>	整型 想要读取的资源id

例子:

+LWM2MREAD: 0,3306,111,5750

3.4.6 +LWM2MWRITE

表示 LwM2M 客户端收到的写请求的主动上报消息。

+LWM2MWRITE:<clientId>,<objectId>,<instanceId>,<num>[,<resId>,<type>,<length>,<valueStr>]^{num}

参数

<clientId>	整型 +LWM2MCREATE指令返回的LwM2M客户端序号
------------	-----------------------------------

<objectId>	整型
	想要写的对象id
<instanceId>	整型
	想要写的实例id
<num>	整型
	想要写的资源的数目
<resId>	整型
	想要写的资源id
<type>	字符型
	"S" 字符型
	"O" 不透明型
	"I" 整型
	"F" 浮点型
<length>	整型
	数值长度
<valueStr>	字符型
	数值，十六进制数表示

例子:

+LWM2MWRITE: 0,3306,0,1,5750,O,4,"54595045"

3.4.7 +LWM2MEXECUTE

表示 LwM2M 客户端收到的执行请求的主动上报消息。

+LWM2MEXECUTE:<clientId>,<objectId>,<instanceId>,<resId>,<length>,< valueStr >

参数

<clientId>	整型
	+LWM2MCREATE指令返回的LwM2M客户端序号
<objectId>	整型
	要执行的对象id
<instanceId>	整型
	要执行的实例id
<resId>	整型
	要执行的资源id
<length>	整型
	数值长度
<valueStr>	字符型

数值，十六进制数表示

例子:

+LWM2MEXECUTE: 0,3303,0,5605,2,"474f"

3.4.8 +LWM2MOBSERVE

表示 LwM2M 客户端收到的观察请求的主动上报消息。

+LWM2MOBSERVE:<clientId>,<oper>,<objectId>,<instanceId>,<resId>

参数

<clientId>	整型
	+LWM2MCREATE指令返回的LwM2M客户端序号
<oper>	整型
	0 观察
	1 取消观察
<objectId>	整型
	要观察的对象id
<instanceId>	整型
	要观察的实例id
<resId>	整型
	要观察的资源id

例子:

观察 3306/111/5750

+LWM2MOBSERVE: 0,0,3306,111,5750

取消观察 3306/111/5750

+LWM2MOBSERVE: 0,1,3306,111,5750

3.4.9 AT+LWM2MREADCONF

该指令回复 lwM2M 服务器的读请求

设置指令 AT+LWM2MREADCONF=<clientId>,<objectId>,<instanceId>,<resId>,<valuetype>,<valuelen>,<value>	回复: OK 如果发生错误, 回复: +LWM2M ERROR: <err>
测试指令 AT+LWM2MREADCONF=?	回复: +LWM2MREADCONF: (list of supported<clientId>),(range of supported<objectId>),(range of

	supported<instanceId>),(range of supported<resId>),(range of supported<valuetype>),(range of supported<valuelen>), "<value>" OK
--	---

参数

<clientId>	整型 +LWM2MCREATE指令返回的LwM2M客户端序号
<objectId>	整型 对象id
<instanceId>	整型 实例id
<resId>	整型 资源id
<valuetype>	整型；数值类型 0 字符型 1 不透明型 2 整型 3 浮点型 4 布尔型
<valuelen>	整型 数值长度
<value>	字符型 数值，如果数值类型是不透明型，数值用十六进制字符串表示

例子：

字符型：

AT+LWM2MREADCONF=0,3306,0,5750,0,5,"hello"

不透明型：

AT+LWM2MREADCONF=0,6,0,4,1,5,"3432383330"

整型：

AT+LWM2MREADCONF=0,3306,0,5851,2,3,"206"

浮点型：

AT+LWM2MREADCONF=0,3303,0,5601,3,4,"3.14"

布尔型：

AT+LWM2MREADCONF=0,3306,0,5850,4,1,"1"

3.4.10 AT+LWM2MWRITECONF

该指令回复 LwM2M 服务器的写请求

设置指令 AT+LWM2MWRITECONF=<clientId>,<result>	回复: OK 如果发生错误, 回复: +LWM2M ERROR: <err>
测试指令 AT+LWM2MWRITECONF=?	回复: +LWM2MWRITECONF: (list of supported<clientId>),(range of supported<result>) OK

参数

<clientId>	整型 +LWM2MCREATE指令返回的LwM2M客户端序号
<result>	整型; 写请求的结果
0x44	写成功, 十进制为68
0x8c	指令超时, 十进制为140
0x84	无此对象, 十进制为132
详细参考 The Constrained Application Protocol (CoAP) rfc 7252	

例子:

AT+LWM2MWRITECONF=0,68

3.4.11 AT+LWM2MEXECUTECONF

该指令回复 LwM2M 服务器的执行请求

设置指令 AT+LWM2MEXECUTECONF=<clientId>,<result>	回复: OK 如果发生错误, 回复: +LWM2M ERROR: <err>
测试指令 AT+LWM2MEXECUTECONF=?	回复: +LWM2MEXECUTECONF: (list of supported<clientId>),(range of supported<result>) OK

参数

<clientId>	整型 +LWM2MCREATE指令返回的LwM2M客户端序号
<result>	整型; result of write command
0x44	执行成功, 十进制为68

0x8c	指令超时，十进制为140
0x84	无此对象，十进制为132
详细参考 The Constrained Application Protocol (CoAP) rfc 7252	

例子:

AT+LWM2MEXECUTECONF=0,68

3.4.12 AT+LWM2MNOTIFY

通知服务器资源有更新

设置指令 AT+LWM2MNOTIFY=<clientId>,<objectId>,<instanceId>,<resId>,<valuetype>,<valuelen>,<value>	回复: OK 如果发生错误, 回复: +LWM2M ERROR: <err>
测试指令 AT+LWM2MNOTIFY=?	回复: +LWM2MNOTIFY: (list of supported<clientId>),(range of supported<objectId>),(range of supported<instanceId>),(range of supported<resourceId>),(range of supported<valuetype>),(range of supported<valuelen>),"<value>" OK

参数

<clientId>	整型
	+LWM2MCREATE指令返回的LwM2M客户端序号
<objectId>	整型
	对象id
<instanceId>	整型
	实例id
<resId>	整型
	资源id
<valuetype>	整型; 数据类型
0	字符串型
1	不透明型
2	整型
3	浮点型
4	布尔型
<valuelen>	整型
	数据长度
<value>	字符型

数据内容，如果数据是不透明型，内容为16进制字符格式

例子：

AT+LWM2MNOTIFY=0,3303,0,5601,3,4,"3.14"

3.4.13 AT+LWM2MUPDATE

该指令更新注册信息，可选择是否更新对象信息。

设置指令 AT+LWM2MUPDATE=<clientId>[,<withobj>]	回复： OK 如果发生错误， 回复： +LWM2M ERROR: <err>
测试指令 AT+LWM2MUPDATE=?	回复： +LWM2MUPDATE: (list of supported<clientId>),(list of supported<withobj>) OK

参数

<clientId>	整型 +LWM2MCREATE指令返回的Lwm2m客户端序号
<withobj>	整型 0 不更新对象信息 1 更新对象信息

3.4.14 <err>说明

<err>	说明
ERROR	输入了错误的指令名，例如：包含中文字符
PARAMETER ERROR	参数错误，例如：超出范围
CANNOT CREATE SEMPH	无法创建信号量
CONFIG ERROR	配置 lwm2m 客户端错误
NO FREE CLIENT	没有多余的客户端，目前只支持一个客户端
OPERATION NO SUPPORT	不支持的操作，例如：读取命令
NO FIND CLIENT	未找到客户端
ADD OBJECT FAILED	添加对象失败
NO FIND OBJECT	未找到该对象
DELETE OBJECT FAILED	删除对象失败
NETWORK NOT READY	网络未准备好，还不能使用数据业务
INTERNAL ERROR	内部错误
REGISTER FAILED, BAD REQUEST	错误请求

SERVER REGISTER REJECT	服务器注册拒绝
REGISTER FAILED, BAD PARAMETER	注册时错误的参数
REGISTER TIMEOUT	注册超时
SESSION INVALID	会话不可用
OBJECT ALREADY ADD	对象已添加

注：目前仅支持 string 形式错误描述，后续考虑支持错误码

4. 附录

4.1 上电检查流程

- (1) AT //判断模组是否上电开机成功
- (2) AT+CFUN=1 //关闭飞行模式
- (3) AT+CEREG? //判断 PS 域附着状态，第二个参数为 1 或 5 表示附着正常

4.2 模组侧设备登陆流程

- (1) 查询leshan.eigencomm.com域名是否能够解析
AT+ECDNS= leshan.eigencomm.com
如果返回IP地址，说明域名解析正常，可以使用域名创建lwm2m实例，即步骤2；如果返回error，说明当前域名解析服务器无法工作，可以直接使用IP地址创建lwm2m实例，先在PC上获取leshan.eigencomm.com的IP地址，得到47.103.106.61，然后使用AT+LWM2MCREATE="47.103.106.61",5683,56830,"ec-lient-128",60
- (2) AT+LWM2MCREATE="Leshan.eigencomm.com",5683,56830,"ec-lient-128",60 //创建实例并注册
- (3) AT+LWM2MADDOBJ=0,3303,0,4,"5601;5602;5700;5701" //添加 Object, Instance 及 Resource

4.3 读写执行流程

- (1) AT+LWM2MREADCONF=0,3303,0,5601,3,4,"3.14" //读操作回复
- (2) AT+LWM2MWRITECONF=0,68 //写操作回复
- (3) AT+LWM2MEXECUTECONF=0,68 //执行操作回复
- (4) AT+LWM2MNOTIFY=0,3303,0,5601,3,4,"3.14" //notify 操作
- (5) AT+LWM2MUPDATE=0 //更新操作

4.4 客户端侧注销流程

- (1) AT+LWM2MDELETE=0 //客户端实例销毁

5. 版本

版本	日期	备注
Draft	2019-4-1	
V1.1	2019-7-23	更新 LwM2M 服务器地址
V1.2	2019-10-12	修改 AT+LWM2MCREATE 的参数类型
V1.3	2020-08-04	增加域名解析查询

6. 关于我们

上海移芯通信科技有限公司坐落于上海张江，公司于 2017 年 2 月成立，从事蜂窝移动通信芯片的研发和销售。公司产品主要应用于广域物联网通信领域，致力于设计最具性价比的蜂窝物联网芯片。公司团队在蜂窝通信芯片上有着辉煌历史和丰富经验。公司所有核心技术全部自研，包括算法&架构、射频、基带、SoC、协议栈软件、平台&应用软件和硬件方案等。

移芯通信首款自主研发的超低功耗 NB-IoT 芯片 EC616 于 2019 年中量产，被绝大部分头部模组企业采用，现已大批量应用于全国各区域各行业。下一代超高集成度 NB-IoT 芯片 EC616S 于 20 年底量产，超低功耗 4G Cat.1 芯片 EC618 工程样片阶段，5G 芯片正在研发中。

上海移芯通信科技有限公司

地址：中国上海市浦东新区祥科路 298 号佑越国际 1 幢 7 层

邮编：201210

电话：

电邮：

网址：<http://www.eigencomm.com>

技术支持窗口

电邮：support@eigencomm.com